🛒 Legacy eCommerce Platform

.NET 4.7 MVC + Razor + jQuery

🗓️ Framework: .NET Framework 4.7 · ⚙️ ASP.NET MVC 5 · 📄 Razor View Engine (.cshtml) · ✨ jQuery 3.4.1 (legacy) · 🗄️ SQL Server Express · 💾 InProc Session State

Complete technical specification for a classic, monolithic eCommerce application built on .NET 4.7 with **Razor templates (.cshtml)** for server-side rendering and **jQuery** for client-side interactivity. The database layer relies on **SQL Server Express**, with heavy reliance on server-side **Session State** for data persistence across requests.

📐 Application Architecture

Pattern

- `ASP.NET MVC 5` (Model-View-Controller)
- **Razor View Engine** (.cshtml templates)
- Service Layer + Repository pattern
- Unity DI Container (legacy)

Project Structure

- `Ecommerce.Web` – MVC frontend, Razor views
- `Ecommerce.Core` – Domain models, interfaces
- `Ecommerce.Data` – EF 6.x DbContext, repositories
- `Ecommerce.Services` – Business logic

🧱 **Legacy characteristic:** The app uses synchronous server-side rendering with Razor templates generating full HTML pages. jQuery enhances UX via AJAX for cart updates, product filtering, and checkout steps without SPA frameworks. Partial views are rendered server-side and returned as HTML fragments.

💾 Session State Management (Legacy Core)

State Persistence

- Heavy reliance on `HttpContext.Session` for temporary data
- `InProc` session state mode (single server deployments)
- No distributed cache (Redis) utilized in default setup

Session Usage Scenarios

- Guest shopping cart storage (`Session["CartItems"]`)
- Multi-step checkout wizard state persistence
- Caching expensive category hierarchy queries
- Temporary UI notifications (via `TempData` backing)

💡 **Legacy Session Architecture:** Utilizing InProc session state means IIS application pool recycles will drop all active guest carts and checkout progress. This is a deliberate hallmark of this specific legacy monolithic design.

📄 Razor Templates – Detailed Specification

Razor View Structure

- `~/Views/Shared/_Layout.cshtml` – Master layout
- `~/Views/Shared/_Header.cshtml` – Navigation partial
- `~/Views/Shared/_Footer.cshtml` – Footer partial
- `~/Views/Shared/_MiniCart.cshtml` – Cart dropdown
- `~/Views/Shared/_ProductCard.cshtml` – Reusable product tile

Razor Features Used

- `@model` directives (strongly-typed views)
- `@Html.Partial()` for nested partials
- `@Html.Action()` for child actions
- `@section Scripts { }` for page-specific JS
- `@RenderSection()` in layout
- HTML Helpers: `@Html.TextBoxFor()` , `@Html.DropDownListFor()`

🖼️ Layout Template Hierarchy

```
~/Views/Shared/_Layout.cshtml
├── _Header.cshtml (logo, search, mini cart)
├── @RenderBody() (main content area)
├── _Sidebar.cshtml (category menu, filters)
└── _Footer.cshtml

~/Views/Shared/_Layout.cshtml structure:
<!DOCTYPE html>
<html>
<head>
    <title>@ViewBag.Title - Legacy Store</title>
    @Styles.Render("~/Content/css")
    @RenderSection("Styles", required: false)
</head>
<body>
    @Html.Partial("_Header")
    <div class="container">
        @RenderBody()
    </div>
    @Html.Partial("_Footer")
    @Scripts.Render("~/bundles/jquery")
    @Scripts.Render("~/bundles/bootstrap")
    @RenderSection("Scripts", required: false)
</body>
</html>
```

🧩 Razor Partial Views & AJAX Integration

Product Listing – Razor + jQuery Pattern

- **Main View:** `~/Views/Product/Index.cshtml` – contains filter form and empty `<div id="product-grid">`
- **Partial View:** `~/Views/Product/_ProductList.cshtml` – renders the grid of `_ProductCard.cshtml` items
- jQuery intercepts filter form submit, sends AJAX GET to `/Product/Filter`
- Controller action returns `PartialView("_ProductList", model)`
- Success callback: `$('#product-grid').html(response)` replaces content

Mini Cart – Razor Child Action

- Layout calls: `@Html.Action("MiniCart", "Cart")`
- `CartController.MiniCart()` returns `PartialView("_MiniCart", cartModel)`
- After AJAX add-to-cart, jQuery refreshes: `$('#mini-cart-container').load('/Cart/MiniCart')`
- Partial view uses `@model CartViewModel` with item count and total

📝 Razor Template Examples

```
@* _ProductCard.cshtml - Reusable product tile *@
@model Ecommerce.Core.Models.Product

<div class="product-card" data-product-id="@Model.Id">
    <a href="@Url.Action("Detail", "Product", new { id = Model.Id })">
        <img src="@Model.ThumbnailUrl" alt="@Model.Name" />
        <h4>@Model.Name</h4>
    </a>
    <span class="price">@Model.Price.ToString("C")</span>
    <button class="add-to-cart-btn btn btn-primary"
            data-product-id="@Model.Id">
        Add to Cart
    </button>
</div>

@* Product/Detail.cshtml - Strongly-typed detail view *@
@model Ecommerce.Core.Models.ProductDetailViewModel
@{
    ViewBag.Title = Model.Product.Name;
}

<div class="product-detail">
    <h1>@Model.Product.Name</h1>
    <div class="gallery">
        @foreach (var image in Model.Images)
        {
            <img src="@image.Url" data-fancybox="gallery" />
        }
    </div>
    <div class="variants">
        @Html.DropDownListFor(m => m.SelectedVariantId,
                              new SelectList(Model.Variants, "Id", "Name"))
    </div>
</div>

@section Scripts {
    <script>
        $(function() {
            $('[data-fancybox]').fancybox();
        });
    </script>
}
```

🗄️ Database – SQL Server Express Details

SQL Server Express Configuration

- Edition: `SQL Server 2016 Express` (or 2019 Express)
- Instance: `.\SQLEXPRESS` (default named instance)
- Authentication: SQL Server Authentication (mixed mode)
- Database name: `LegacyEcommerceDb`
- Max database size: **10 GB** (Express limitation)

Resource Constraints

- RAM utilized: max ~1.4 GB (Express buffer pool limit)
- CPU: 1 socket / 4 cores (Express limitation)
- No SQL Server Agent (manual backups via scripts)
- Full-Text Search available (if Advanced Services installed)

⚠️ **Express edition constraints:** The database size cannot exceed 10 GB. The legacy eCommerce app typically uses around 3-6 GB for catalog, orders, and customer data. Regular maintenance (index rebuild, archiving old orders) is required.

🧩 Core eCommerce Modules

Product Catalog

- Razor views: `Index.cshtml` , `Detail.c`
- Category tree (nested, lazy-loaded via jQuery AJAX)
- jQuery-based image gallery (Fancybox)

Shopping Cart

- Razor partials: `_MiniCart.cshtml` , `Ca`
- Session-based cart (server `HttpContext.Session`)
- jQuery AJAX add-to-cart (no page refresh)

Checkout Flow

- Razor views per step: `Address.cshtml` , `Shippi`
- Multi-step wizard (jQuery steps/tabs, bounded by Session State)
- Client-side validation (jQuery Validate + Unobtrusive)

User Account

- Razor views: `Login.cshtml` , `Register.cshtml` , `Orders.cshtml`
- ASP.NET Identity 2.0 (OWIN, cookie auth)
- Order history with DataTables jQuery plugin

💻 jQuery Integration & Frontend Stack

📦 Core Libraries

Razor View Engine · jQuery 3.4.1 · jQuery UI 1.12.1 · jQuery Validate · jQuery Unobtrusive Ajax · DataTables · Fancybox 3

⚡ jQuery + Razor Patterns

- `$.ajax()` loads Razor partial views as HTML
- `$('#container').load('/controller/action')`
- Anti-forgery token in AJAX headers
- `@section Scripts { }` for page-specific jQuery

🗃️ Entity Framework & SQL Express Integration

Entity Framework 6.2 (Database First with SQL Express)

- EDMX model generated from existing SQL Express database
- Key entities: `Product` , `Category` , `CartItem` , `Order` , `Customer`
- Connection string uses `.\SQLEXPRESS` instance
- Provider: `System.Data.SqlClient`

```
<connectionStrings>
  <add name="EcommerceDb"
       connectionString="Server=.\SQLEXPRESS; Database=LegacyEcommerceDb;
       User Id=legacy_app_user; Password=****; MultipleActiveResultSets=True;"
       providerName="System.Data.SqlClient" />
</connectionStrings>
```

🔐 Security & Anti-Forgery

CSRF Protection

- `@Html.AntiForgeryToken()` in all Razor forms
- jQuery global AJAX header setup
- `[ValidateAntiForgeryToken]` attribute

Authentication

- OWIN cookie authentication
- `[Authorize]` attribute on controllers
- Razor conditional rendering: `@if (User.Identity.IsAuthenticated)`

📦 NuGet Packages & Dependencies

Microsoft.AspNet.Mvc 5.2.7 · Microsoft.AspNet.Razor 3.2.7 · EntityFramework 6.4.4 · Microsoft.AspNet.Identity.Core · jQuery 3.4.1 · Microsoft.jQuery.Unobtrusive.Ajax · jQuery.Validation · Bootstrap 3 · Newtonsoft.Json 12.0.3

⚙️ Configuration & Web.config Highlights

```
<system.web>
  <compilation debug="false" targetFramework="4.7" />
  <httpRuntime targetFramework="4.7" maxRequestLength="30720" />
  <!-- Critical Legacy Configuration: InProc Session State -->
  <sessionState mode="InProc" timeout="25" />
  <authentication mode="None" /> <!-- OWIN handles auth -->
</system.web>
<appSettings>
  <add key="webpages:Version" value="3.0.0.0" />
  <add key="webpages:Enabled" value="false" />
</appSettings>
```

🧪 Testing & Maintenance Notes

- 📄 Razor views are compiled at runtime (no precompilation by default in legacy setup)
- ⚡ jQuery dependent; manual testing with browser dev tools
- 📋 Bundling via `BundleConfig.cs` using `@Scripts.Render()`
- 🔄 IIS hosted (Windows Server 2016) with .NET CLR v4.0 app pool
- 🗄️ SQL Express maintenance: weekly index rebuild, manual backups
- ⚠️ Known issues: jQuery version conflicts; Razor compilation errors only visible at runtime

🗓️ Last stable release: Q2 2020 · 📄 Razor .cshtml templates · 🗄️ SQL Express 10GB limit · 💾 Session State Backed

🔗 Legacy eCommerce · .NET 4.7 MVC · Razor Views · jQuery · SQL Server Express

Specification document — covers Razor templates, jQuery patterns, checkout, catalog, and SQL Server Express infrastructure. For migration reference only.